Drawing AR Quivers of Finite-Dimensional Algebras with GAP

Quick Start

Launch the container

Automatic launch: click 

, or open one of the following links:

- Stable version: https://mybinder.org/v2/gh/zhangchenchengSJTU/GAP_AR_Quiver/main
- Testing version: https://mybinder.org/v2/gh/zhangchenchengSJTU/GAP_AR_Quiver/test

Manual launch: open Binder and choose the GitHub repository option. Complete only the following steps.

1. Make sure that the `GitHub repository name or URL` field is set to `GitHub`. Paste `https://github.com/zhangchenchengSJTU/GAP_AR_Quiver` into the box on the right.
2. Click `Launch` directly.
3. Wait until the environment is ready. Binder will then redirect you to the Jupyter Notebook page.

Enter Jupyter Notebook

After entering Jupyter Notebook, the browser address should look like `https://hub.bids.mybinder.org/user/zhangchenchengsjtu-gap_ar_quiver-???????/treee`. In the root directory, you should see the following three items:

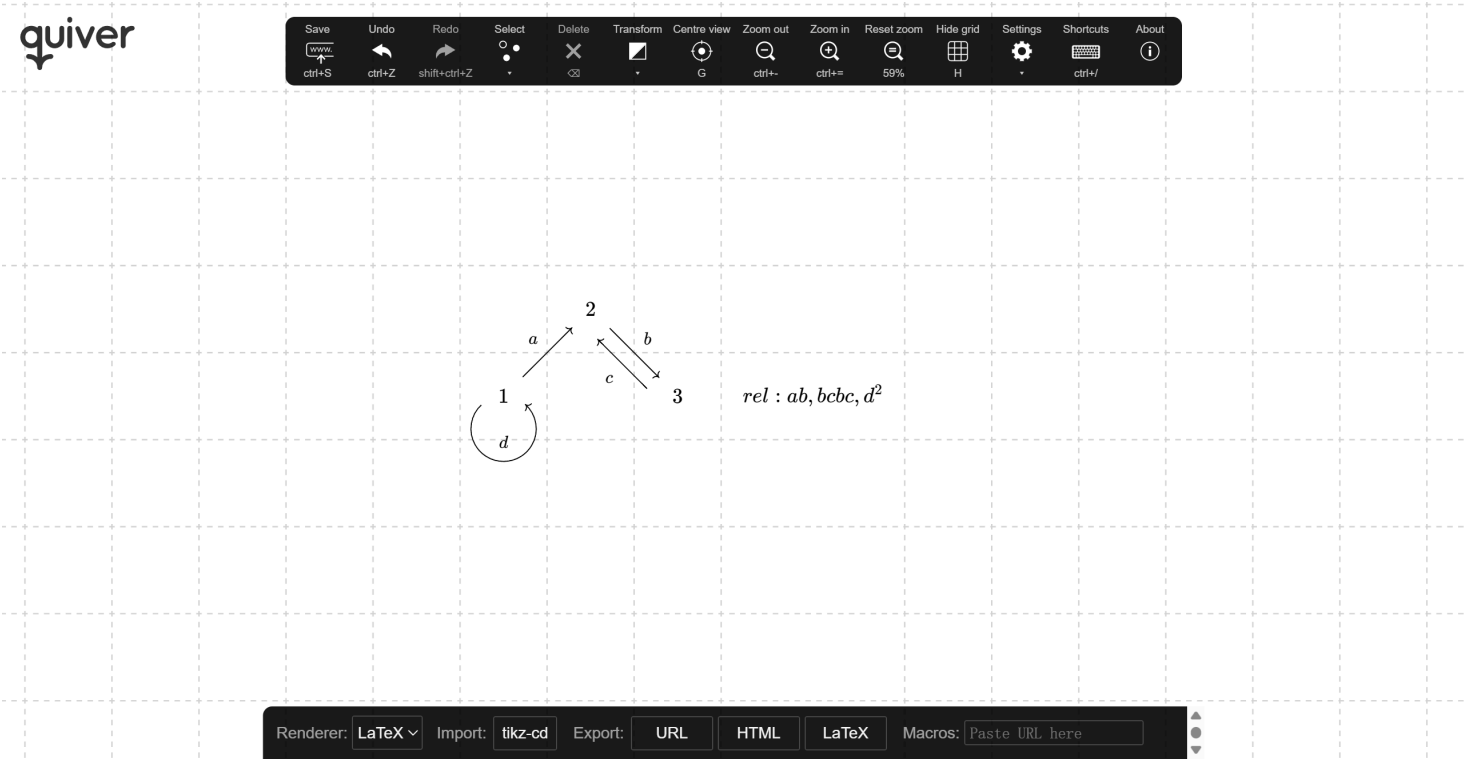
- `Dockerfile`: the environment specification, mainly for developers. Users do not need to edit it.
- `Readme.md`: this documentation file.
- `ARquiver`: the working directory for drawing AR quivers. After entering this folder, you will see:
  - `source`: the source-code directory. Users usually do not need to inspect it.
  - `run.ipynb`: the notebook used to run the computation and rendering scripts.
  - `example.txt`, `untitled.txt`, and similar files: input files containing quiver data.

Draw the path algebra

Use `https://q.uiver.app/` to draw a quiver with relations. Please follow these conventions:

- vertices of the quiver should be positive integers;
- arrows should be simple Latin letters or Greek letters written in $\LaTeX$ format, such as `a` or `\alpha`;
- choose an empty grid cell and enter the relations of the path algebra in the form `rel: .....`

Example:



Click `LaTeX` at the bottom of the q.uiver page and copy the generated `tikzcd` source code.

Create a new `txt` file named `yourfile.txt` inside the `ARquiver` directory, paste the copied source code into it, save the file, and close it.

Draw the AR quiver

Open `run.ipynb` and run the following cells in order:

```
# Compute algebra data: filename.txt -> filename.log
%run source/compute_all.py
```

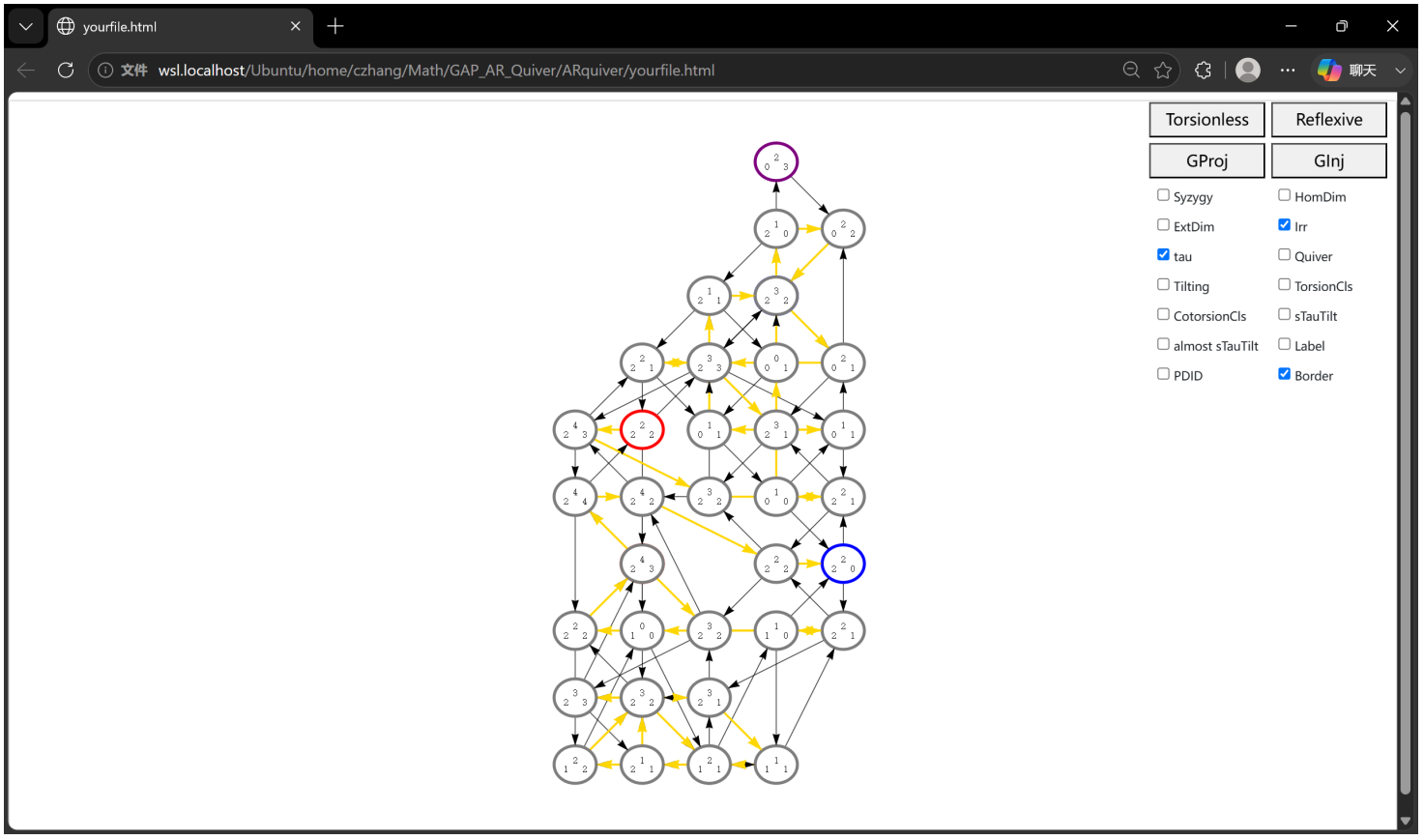
```
# Render interactive diagram: filename.log -> filename.html
%run source/render_all.py
```

The `ARquiver` directory will then contain:

- `yourfile.log`: the algebra computation log;
- `yourfile.html`: the interactive AR-quiver canvas.

Arrange the AR quiver into a standard form

Open `yourfile.html` to view the interactive visualization of the AR quiver.



We first describe the basic AR-quiver controls.

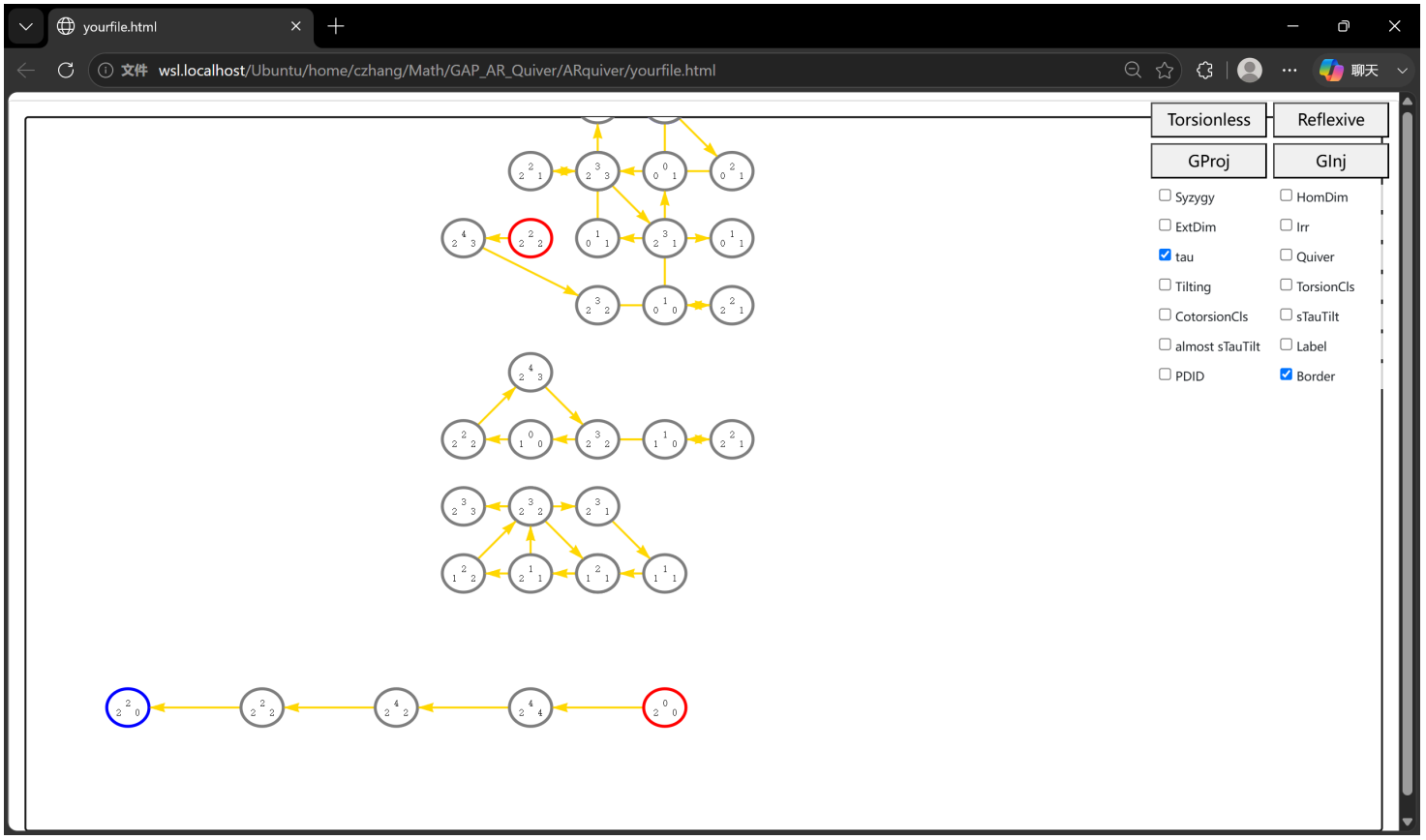
- Each ellipse represents an indecomposable module. Its dimension vector is arranged according to the vertex positions in the original `tikzcd` input.
- Purple vertices are both projective and injective. Red vertices are projective but not injective, while blue vertices are injective but not projective.
- `Irr`: show or hide irreducible morphisms, drawn as black arrows.
- `tau`: show or hide the AR translation $\tau = D\mathrm{Tr}$, drawn as golden arrows.
- `Border`: show or hide vertex borders.
- `Ctrl + Z` undoes an operation, and `Ctrl + Y` redoes it.

The main manual task is to arrange the AR quiver into a readable standard form. Here are some useful guidelines.

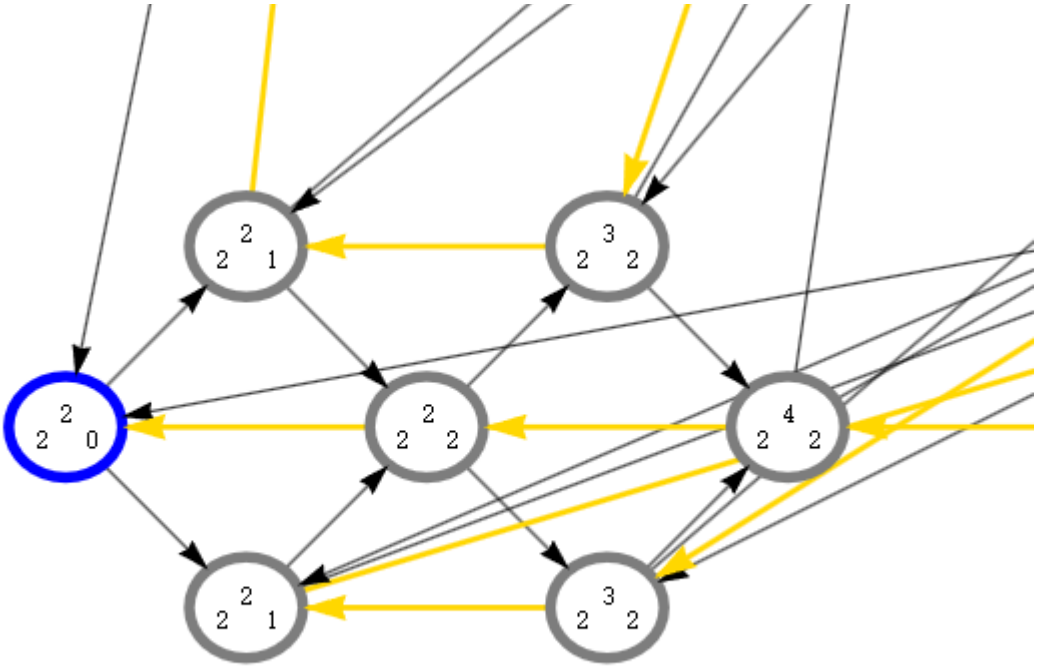
Lemma. The τ-orbits, namely the orbits of the golden arrows, are disjoint. Projective-injective objects do not belong to any τ-orbit. Every other indecomposable module belongs to exactly one τ-orbit. Hence each τ-orbit is one of the following two types:

- a straight path from an injective object to a projective object;
- a cycle that contains no projective or injective object.

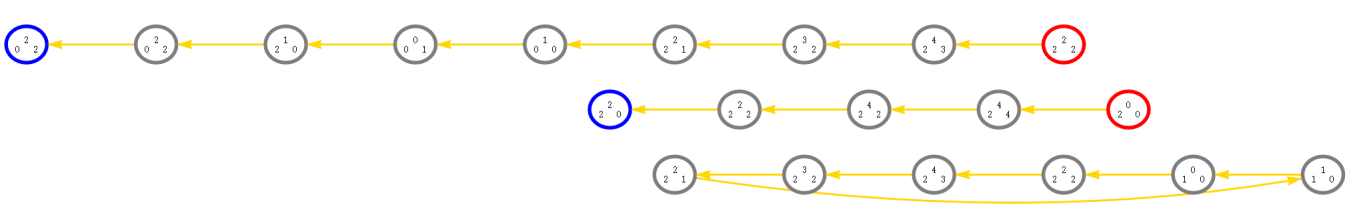
Following this lemma, we first arrange the projective object $_2{}^2{}_0$. Turn off `Irr`, then place the arrow $_2{}^2{}_0 \leftarrow {}_2{}^2{}_2$ horizontally in an empty region. Long-press the golden arrow to align the next arrow. This gives:



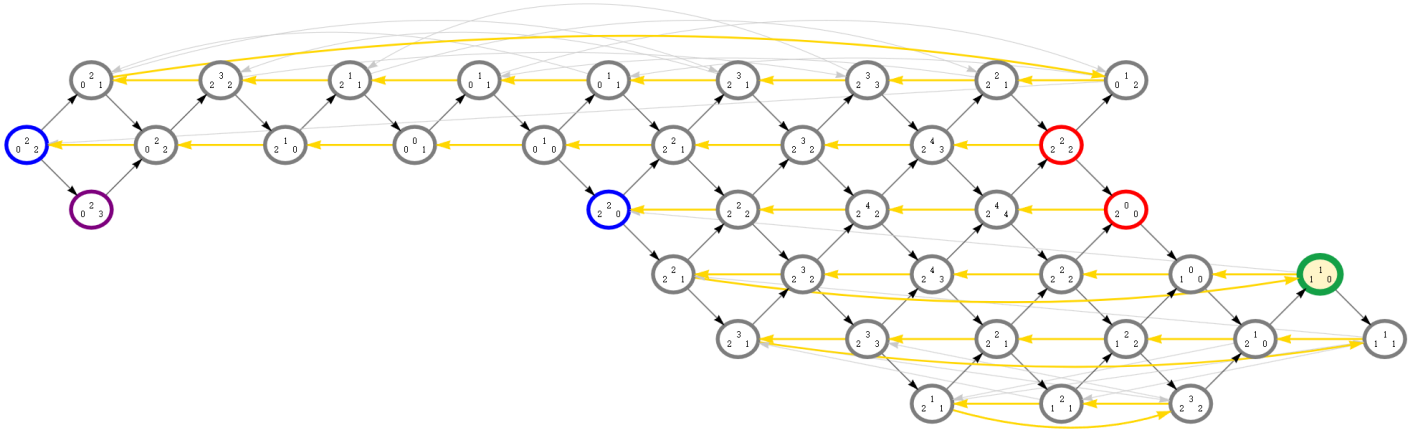
Next, turn on `Irr` and look for the almost split short exact sequence $_2{}^2{}_0 \rightarrowtail \bigoplus M_i \twoheadrightarrow {}_2{}^2{}_2$.



Then turn off `Irr` again and long-press the golden arrows for alignment. If a cycle appears, select an edge and use the ↑ and ↓ keys to adjust the curvature of the arrow. After a sequence of such operations, one obtains the τ-orbits:



Finally, turn off `tau` and adjust the curvature of the horizontal black arrows that were hidden behind the golden arrows. If some edges are visually inconvenient, double-click them to switch between dark and light colors. The final AR quiver is then obtained:



Features

The generated HTML file is an interactive workspace for exploring the representation theory of the input algebra. It combines the AR quiver, homological data, torsion-theoretic structures, and several highlighting tools in a single browser page.

Navigation and editing

- Drag vertices to arrange the AR quiver manually.
- Drag the canvas to move the view, and use the mouse wheel to zoom.
- Use `Fit` to refit the current diagram into the viewport.
- Use `Ctrl + Z` and `Ctrl + Y` to undo and redo layout and coloring operations.
- Double-click an edge to toggle between dark and light colors.
- Select curved edges and use the arrow keys to adjust their curvature.
- Use `Export TeX` to export the current AR-quiver layout as TeX code.

Labels and local information

The controls provide several ways to inspect individual indecomposable modules.

- `Label` displays the internal labels of indecomposable modules, which are useful for comparing the canvas with `yourfile.log`.
- Hovering over a vertex also reveals its module label and additional stored information.
- `PDID` displays projective and injective dimensions. The value `-1` denotes infinity.
- `TopSoc` displays top and socle information when available.
- The original quiver of the algebra can be opened as a small draggable window. The `Open in q.uiver` button opens the q.uiver URL stored in the first line of the corresponding `txt` file.

Color legend and visual conventions

The `Color legend` panel summarizes the visual meaning of the colors used in the page.

- Projective, injective, and projective-injective modules are indicated by vertex borders.
- Torsionless, reflexive, Gorenstein projective, and Gorenstein injective modules can be highlighted directly on the AR quiver.
- Irreducible morphisms, AR translation arrows, syzygy arrows, cosyzygy arrows, Hom-dimension edges, and Ext-dimension edges use distinct colors.
- Floating labels indicate projective dimension, injective dimension, top, and socle data.

Homological and module-class tools

Several buttons highlight important classes of indecomposable modules.

- `Torsionless` highlights non-projective indecomposable torsionless modules, i.e. submodules of projective modules, equivalently modules for which the canonical map $M \to DDM$ is injective.
- `Reflexive` highlights non-projective indecomposable reflexive modules, i.e. modules for which the canonical map $M \to DDM$ is an isomorphism.
- `GProj` highlights non-projective indecomposable Gorenstein projective modules.
- `GInj` highlights non-injective indecomposable Gorenstein injective modules.
- `Syzygy` displays arrows related to kernels of projective covers.
- `Cosyzygy` displays the corresponding cosyzygy information.
- `HomDim` and `ExtDim` display edges encoding nonzero dimensions of $\mathrm{Hom}(M, N)$ and $\mathrm{Ext}^1(M, N)$, respectively.

For detailed numerical data, it is often useful to compare the visualization with the corresponding `yourfile.log` file.

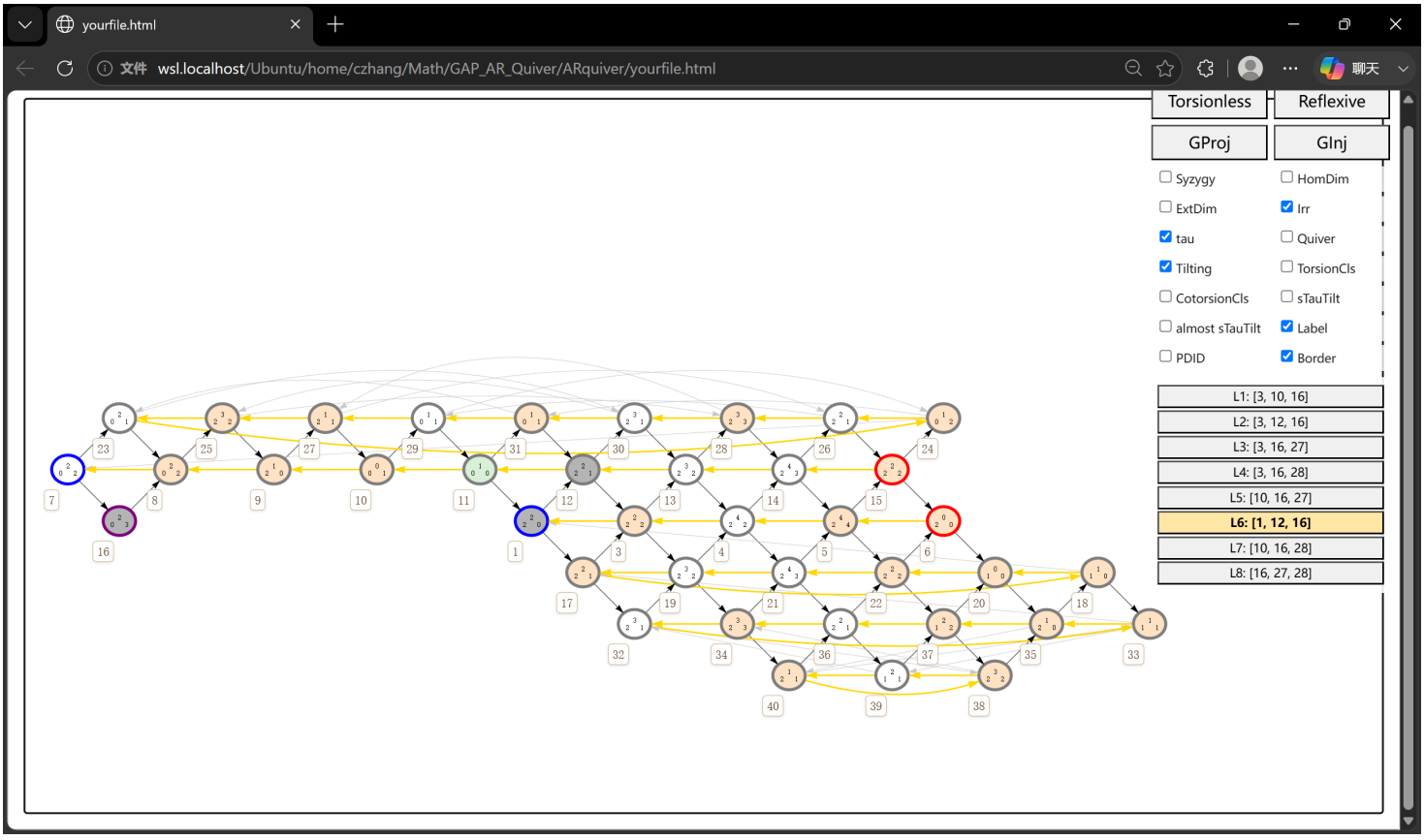
Tilting modules and torsion theories

The page can display classical tilting modules and the torsion-theoretic structures induced by them. We use the standard definition: a module T is tilting if it satisfies the following conditions:

- $\mathrm{pd}\, T \leq 1$;
- $\mathrm{Ext}^{\geq 1}(T, T) = 0$;
- there exists a short exact sequence $A \rightarrowtail T^0 \twoheadrightarrow T^1$ with $T^0, T^1 \in \mathrm{add}(T)$, where A is the path algebra.

When a tilting module T is selected:

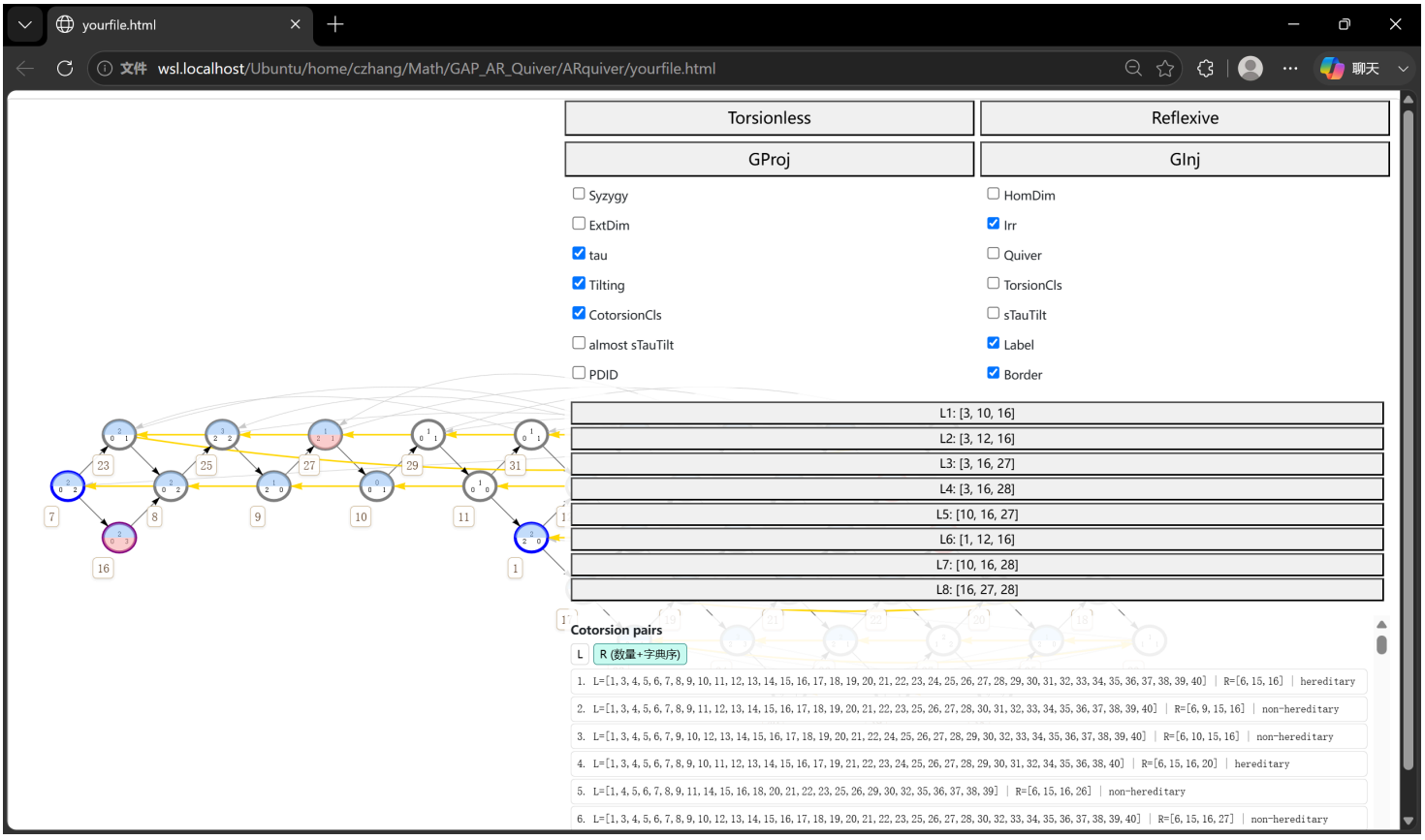
- grey vertices indicate the indecomposable direct summands of T;
- red and grey vertices indicate the torsion class $\mathrm{gen}(T) = \mathrm{Ker}\, \mathrm{Ext}^1(T, -)$;
- green vertices indicate the torsion-free class $\mathrm{Ker}\, \mathrm{Hom}(T, -)$.



`TorsionCls` lists torsion theories. After selecting one, the torsion class and torsion-free class are shown by red and green highlights.

`CotorsionCls` lists cotorsion theories. After selecting one, the left and right classes are shown by blue and red highlights.

- Some objects may belong to both sides of a cotorsion theory; in that case, half-coloring is used.
- Use `Ctrl + L` to enter or leave full-screen mode.
- Double-click `L` or `R` to change the sorting order of cotorsion theories.



Support τ-tilting modules

`sTauTilt` displays support τ-tilting modules. A pair (P, M) is a support τ-tilting module if:

- $\mathrm{Hom}(M, \tau M) = 0$, so M is rigid;
- $\mathrm{Hom}(P, M) = 0$, where P is projective;
- the total number of indecomposable direct summands of P and M is n, the number of vertices of the path algebra.

With an appropriate sorting order, the support τ-tilting data can be used to inspect all cluster-tilting objects in this setting, equivalently the maximal rigid objects in the displayed category.

`almost sTauTilt` lists almost support τ-tilting modules.

Acknowledgement

The author gratefully acknowledges the developers and maintainers of GAP, Binder, and q.uiver for providing essential tools and infrastructure used by this project. The implementation was inspired in part by A. Konovalov's try-gap-in-jupyter repository. The author also acknowledges the assistance of the AI model `chatgpt-5.5-thinking` during the development and refinement of the codebase.